TALLER

Análisis de Vulnerabilidades OWASP

Seguridad del Software

Nombre: Jose Luis Lancheros Ayora

Fecha: 28 de octubre de 2026

Objetivos del taller

- Identificar vulnerabilidades del OWASP Top 10 en escenarios reales.
- Analizar el impacto de cada vulnerabilidad sobre la confidencialidad, integridad y disponibilidad.
- Proponer contramedidas técnicas concretas.
- Aplicar criterio de priorización de riesgos con base en probabilidad e impacto.

CASO DE ESTUDIO

VaultApp es una startup de 3 años que ofrece una plataforma web SaaS para gestión segura de contraseñas corporativas, almacenamiento de documentos sensibles y autenticación centralizada de empleados. Atiende a 80 clientes corporativos en Colombia y está en proceso de expansión hacia Latinoamérica. Su propuesta de valor central es la seguridad, lo que hace que cualquier vulnerabilidad tenga un impacto reputacional y legal especialmente grave.

Arquitectura del sistema:

Componente	Tecnología	Descripción
Frontend	React 18	SPA con rutas protegidas por JWT almacenado en localStorage
Backend API	Node.js + Express	REST API sin rate limiting
Base de datos	PostgreSQL 11.2	Sin cifrado en reposo. Contraseñas almacenadas con MD5
Autenticación	JWT (HS256)	Token con expiración de 30 días. Sin rotación de secretos

Servidor	Ubuntu 18.04 LTS	Sin actualizaciones desde hace 14 meses. Puertos 22 y 8080 abiertos al público
Dependencias	npm (sin auditoría)	237 dependencias, 12 con CVEs conocidos según npm audit
Logging	Sin sistema centralizado	Sin alertas ni monitoreo de seguridad

Reporte de incidente reciente — Incidente #2024-047: Un usuario reportó poder acceder a documentos de otras empresas simplemente cambiando el parámetro id en la URL. El equipo de soporte respondió el ticket 6 días después. No hay registros que indiquen cuándo comenzó el problema ni cuántos documentos fueron consultados por personas no autorizadas.

PARTE 1

Mapeo OWASP Top 10

Con base en la descripción del sistema y el incidente reportado, identifiquen qué vulnerabilidades del OWASP Top 10 (2021) están presentes en VaultApp. Para cada una que identifiquen, completen la tabla:

#	Vulnerabilidad OWASP	Código OWASP	¿Dónde se evidencia en el sistema?	Impacto potencial (C/I/A)
1	Broken Access Control	A01:2021	Se ve en el incidente #2024-047. Un usuario cambió el parámetro id de la URL y vio documentos de otras empresas. Eso quiere decir que la API no revisa que el documento sea de la empresa de quien lo pide. Es un IDOR.	C: alto. I: medio. A: bajo. Se leen documentos de otros clientes. Si editar o borrar usan el mismo id, también se podrían alterar.
2	Cryptographic Failures	A02:2021	La base PostgreSQL no tiene cifrado en reposo y las contraseñas están con MD5. MD5 es muy rápido, así que adivinar contraseñas con una copia de la base es fácil. Y lo que se guarda son contraseñas corporativas y documentos sensibles.	C: alto. I: medio. A: bajo. Quien consiga la base o un respaldo lo lee todo. Con las claves que descubra puede entrar y modificar datos.

#	Vulnerabilidad OWASP	Código OWASP	¿Dónde se evidencia en el sistema?	Impacto potencial (C/I/A)
3	Insecure Design	A04:2021	La API REST no tiene rate limiting y los datos de las empresas no están separados desde el diseño, porque con solo un id se llegó a documentos de otro cliente. OWASP pide separar los tenants desde el diseño y limitar el consumo de recursos por usuario.	C: medio. I: bajo. A: alto. Sin límites se pueden probar claves o ids sin parar y también saturar la API.
4	Security Misconfiguration	A05:2021	Los puertos 22 y 8080 del servidor están abiertos al público. El caso no dice qué corre en el 8080 y, si es la API, está expuesta sin nada delante. OWASP cuenta los puertos y servicios innecesarios dentro de esta categoría.	C: alto. I: alto. A: medio. SSH expuesto recibe intentos de fuerza bruta todo el tiempo y quien entre al servidor controla todo.
5	Vulnerable and Outdated Components	A06:2021	Ubuntu 18.04 lleva 14 meses sin actualizaciones y PostgreSQL es la 11.2. Además 12 de las 237 dependencias de npm tienen CVE conocidos y nadie las audita. Ubuntu 18.04 terminó su soporte estándar en mayo de 2023 y PostgreSQL 11 en noviembre de 2023.	C: alto. I: alto. A: alto. Una falla conocida puede dar ejecución remota de código y con eso el atacante hace lo que quiera en el servidor.
6	Identification and Authentication Failures	A07:2021	El JWT dura 30 días, está en localStorage y se firma con un secreto HS256 que nunca se rota. No hay rate limiting en el login, las contraseñas están con MD5 y el caso no menciona MFA.	C: alto. I: alto. A: bajo. Un token robado sirve un mes y, si se filtra el secreto, se pueden fabricar tokens de cualquier usuario.
7	Security Logging and Monitoring Failures	A09:2021	No hay logging centralizado, ni alertas, ni monitoreo de seguridad. Por eso en el incidente no se sabe cuándo empezó el problema ni cuántos documentos se vieron, y soporte tardó 6 días en responder.	C, I y A: indirecto. No causa el daño por sí sola, pero deja que cualquier ataque pase sin que nadie lo vea y no permite medir el alcance.

Usé los códigos del Top 10 de 2021, que es la versión que pide el taller. OWASP ya publicó la edición 2025 y ahí algunos números cambiaron, por ejemplo Injection pasó de A03 a A05.

1. ¿Cuántas vulnerabilidades del Top 10 identificaron en total? ¿Hay alguna que definitivamente NO aplica al caso? Justifiquen.

   Identifiqué 7 de las 10 categorías, que son A01, A02, A04, A05, A06, A07 y A09. La que descarto con seguridad es A10 Server Side Request Forgery, porque el caso no describe ninguna función donde el servidor consulte una dirección que escriba el usuario, y eso es lo que hace falta para un SSRF.

   Con A03 Injection y A08 Software and Data Integrity Failures no tengo evidencia, pero tampoco puedo decir que no apliquen. Para A03 habría que revisar cómo se arman las consultas a PostgreSQL y qué muestra el React con datos de usuarios. Para A08 habría

que ver cómo se instalan las dependencias y cómo se despliega el código. Por eso no las puse en la tabla. Aun así, el JWT en localStorage haría mucho más grave un XSS, que OWASP cuenta dentro de A03.

PARTE 2

Análisis profundo de tres vulnerabilidades

Seleccionen las 3 vulnerabilidades que consideren más críticas para VaultApp y respondan para cada una:

Elegí A01, A02 y A06 porque son las tres que quedaron con impacto crítico en la matriz de la Parte 3. A01 es la única que ya ocurrió, A06 le puede dar al atacante el control de todo el servidor y A02 hace que cualquier filtración de la base sea lo peor que le puede pasar a un gestor de contraseñas. Las otras quedaron con impacto alto o medio.

Vulnerabilidad 1

Campo	Respuesta del equipo
Nombre y código OWASP	Broken Access Control, A01:2021. En este caso es un IDOR, una referencia directa insegura a objetos.
¿Dónde se evidencia en el sistema?	En el incidente #2024-047. Un usuario cambió el parámetro id de la URL y vio documentos de otras empresas, o sea que la API busca el documento por su id y no revisa de quién es. Es la única falla del caso que ya ocurrió en la práctica.
¿Cómo podría un atacante explotarla?	El atacante crea una cuenta o usa la de un cliente, abre un documento suyo y mira el id en la URL. Luego cambia ese id por otros y los prueba uno por uno con un script. Si los ids son consecutivos, como suele pasar, se puede bajar todo lo de los 80 clientes en pocas horas. La API no tiene rate limiting que lo frene y sin logs nadie lo vería.
Impacto sobre Confidencialidad (C)	Alto. Se leen documentos sensibles de cualquier empresa y se rompe la separación entre clientes, que es lo mínimo que se espera de un SaaS.
Impacto sobre Integridad (I)	Medio. Del caso solo se sabe que se pueden ver. Si las rutas para editar o borrar usan el mismo id y tampoco validan, se podrían cambiar o borrar documentos de otras empresas. Habría que probarlo.
Impacto sobre Disponibilidad (A)	Bajo. El servicio seguiría funcionando. Solo se perdería el documento que alguien borre.
¿Qué clientes o datos están en riesgo?	Los 80 clientes corporativos, porque la falla no distingue entre empresas. Los datos en riesgo son los documentos sensibles guardados en la plataforma. No se sabe cuántos se vieron porque no hay registros.

Vulnerabilidad 2

Campo	Respuesta del equipo
Nombre y código OWASP	Cryptographic Failures, A02:2021.
¿Dónde se evidencia en el sistema?	En la base de datos. PostgreSQL 11.2 no tiene cifrado en reposo y las contraseñas se guardan con MD5. OWASP dice que hay que cifrar todos los datos sensibles en reposo y que MD5 es un algoritmo obsoleto. VaultApp guarda justo lo más sensible, contraseñas corporativas y documentos.
¿Cómo podría un atacante explotarla?	Primero necesita llegar a la base o a un respaldo, y en este caso tiene caminos: el servidor sin parches y con puertos abiertos, un respaldo mal protegido o el mismo fallo de ids repetido en más rutas. Con la base en su poder, los hashes MD5 se rompen muy rápido porque una GPU prueba miles de millones por segundo, y si no tienen sal las tablas rainbow resuelven las contraseñas comunes casi al instante. Después prueba esas contraseñas en el login y en otros sitios, porque la gente las repite. Lo que no está cifrado lo lee directamente.
Impacto sobre Confidencialidad (C)	Alto. Salen las contraseñas de los usuarios y todo lo guardado sin cifrar. Para un gestor de contraseñas es el peor daño posible.
Impacto sobre Integridad (I)	Medio. Con las claves descubiertas el atacante inicia sesión como el usuario y puede cambiar o borrar datos.
Impacto sobre Disponibilidad (A)	Bajo. Esta falla no tumba el servicio por sí sola.
¿Qué clientes o datos están en riesgo?	Todos los clientes. Los datos en riesgo son las contraseñas de los empleados de las 80 empresas, las contraseñas corporativas que guardan en la plataforma y los documentos sensibles. Con eso el atacante podría entrar a los sistemas de los clientes y no solo a VaultApp.

Vulnerabilidad 3

Campo	Respuesta del equipo
Nombre y código OWASP	Vulnerable and Outdated Components, A06:2021.
¿Dónde se evidencia en el sistema?	En la tabla de arquitectura. Ubuntu 18.04 lleva 14 meses sin actualizaciones y su soporte estándar terminó el 31 de mayo de 2023. PostgreSQL 11.2 salió en febrero de 2019 y la versión 11 dejó de recibir parches en noviembre de 2023. Además npm audit marca 12 de las 237 dependencias con CVE conocidos y nadie las audita.
¿Cómo podría un atacante explotarla?	Un atacante automatizado escanea Internet buscando versiones viejas. Con los puertos 22 y 8080 abiertos ve qué software corre y busca CVE con exploit público. Por ejemplo, PostgreSQL 11.2 entra en el rango de CVE-2019-10164, que NVD califica con 8,8. Un usuario autenticado cambia su propia contraseña por un valor armado a propósito, desborda un buffer y puede ejecutar código como la cuenta de PostgreSQL. Con las dependencias de npm es más directo, porque si una de las 12 falla en una ruta pública, el ataque puede ser una sola petición.
Impacto sobre Confidencialidad (C)	Alto. Con ejecución remota de código el atacante lee la base, el secreto del JWT y las variables de entorno.
Impacto sobre Integridad (I)	Alto. Puede modificar datos y código de la aplicación o dejar una puerta trasera.
Impacto sobre Disponibilidad (A)	Alto. Puede borrar o cifrar los datos, como en un ransomware, o apagar el servicio.
¿Qué clientes o datos están en riesgo?	Todos. Un servidor comprometido da acceso a los datos de los 80 clientes, a las credenciales guardadas, a los respaldos y al secreto con el que se firman los tokens. Desde ahí también se podría atacar a los clientes.

PARTE 3

Matriz de riesgo

Ubiquen cada vulnerabilidad identificada en la Parte 1 usando el número (#) de la tabla anterior.

Probabilidad ↓ / Impacto →	Bajo	Medio	Alto	Crítico
Alta		#3, #7	#4, #6	#1, #5
Media				#2
Baja				

Pinté la matriz según probabilidad por impacto. Lo que quedó en rojo lo trato como prioridad Alta en el plan y lo naranja como Media. No puse nada en Baja porque todas las que identifiqué tienen evidencia clara en el caso.

1. ¿Qué criterios usaron para estimar la probabilidad? ¿Tuvieron desacuerdos dentro del equipo?

   Para la probabilidad me fijé en cuatro cosas. Si ya pasó o si se puede explotar sin hacer nada especial, como cambiar un número en una URL. Si se llega desde Internet sin tener cuenta, como pasa con los puertos 22 y 8080. Si hay herramientas o exploits públicos que lo hacen automático, como con los componentes viejos. Y si el atacante tiene que encadenar otra falla antes. Por esa última razón puse la A02 en Media, porque primero hay que llegar a la base de datos o a un respaldo.

   Para el impacto miré cuántos clientes se afectan, si lo que se pierde son credenciales o documentos, si el daño se puede devolver y qué consecuencias legales tiene.

   Hice el taller solo, así que no hubo desacuerdos de equipo. La duda que tuve fue con la A09, porque no causa daño por sí sola. La dejé en impacto Medio porque solo hace que los otros ataques duren más y no se pueda medir el daño. Si se mira el incidente #2024-047, donde ni siquiera se sabe qué documentos se vieron, se podría subir a Alto.

2. VaultApp vende seguridad como su propuesta de valor principal. ¿Cambia eso el umbral de riesgo aceptable frente a otra empresa? Justifiquen.

   Sí lo cambia, y lo hace más estricto. Una empresa normal puede aceptar un riesgo medio por un tiempo mientras lo arregla. VaultApp no, por varias razones.

   Su producto es la seguridad, entonces una falla es una falla del producto y no un problema de soporte. Además guarda las contraseñas y los documentos de 80 empresas, así que si cae VaultApp cae la seguridad de todos sus clientes a la vez y el daño llega a sistemas que ni siquiera administra. También hay obligaciones legales. La Ley 1581 de 2012 pide manejar los datos con medidas técnicas, humanas y administrativas que den seguridad a los registros, y avisar a la Superintendencia de Industria y Comercio cuando hay violaciones a los códigos de seguridad. Las multas pueden llegar a 2.000 salarios mínimos mensuales y también existe la suspensión de actividades. Y por último está la confianza, que se pierde rápido y se recupera muy lento, justo cuando se quiere expandir a Latinoamérica, donde cada país tiene su propia ley de datos personales.

   Por eso yo no aceptaría ningún riesgo de la zona roja de la matriz y los de la zona naranja los resolvería en semanas. Solo aceptaría riesgos bajos, con una fecha para volver a revisarlos.

PARTE 4

Plan de remediación

Para cada vulnerabilidad identificada, propongan una contramedida técnica concreta. No escriban "mejorar la seguridad" — escriban exactamente qué cambiarían en el sistema o en el proceso de desarrollo.

#	Vulnerabilidad	Contramedida técnica concreta	Prioridad	Esfuerzo estimado
1	Broken Access Control, A01:2021	Agregar un middleware de autorización que, antes de devolver un documento, compare la empresa del documento con la empresa del usuario que viene en el token, y responda 404 si no coinciden. En todas las consultas filtrar por el id del documento y por el tenant_id del usuario. Activar Row Level Security en PostgreSQL por tenant_id como segunda barrera. Cambiar los ids consecutivos por UUID v4, solo como ayuda extra. Escribir pruebas automáticas donde un usuario de la empresa A pide documentos de la empresa B y debe recibir 404, y repetirlas en todas las rutas que usan ids.	Alta	3 días
2	Cryptographic Failures, A02:2021	Cambiar MD5 por Argon2id, con mínimo 19 MiB de memoria, 2 iteraciones y paralelismo 1, que es lo que recomienda OWASP. Para no esperar a que todos entren, aplicar Argon2id sobre el hash MD5 que ya existe y cambiarlo por un Argon2id de la clave real en el siguiente inicio de sesión de cada usuario. A los usuarios que llevan mucho tiempo inactivos se les borra el hash y se les pide restablecer la clave. Cifrar los discos y los respaldos con LUKS. Cifrar los documentos y las contraseñas guardadas con AES 256 GCM, con una llave distinta por cliente en un KMS, y exigir TLS 1.2 o superior también en la conexión con PostgreSQL.	Alta	3 días el hash y 3 semanas el cifrado
3	Insecure Design, A04:2021	Poner rate limiting con express-rate-limit y Redis. Por ejemplo 5 intentos de login por cuenta y por IP cada 15 minutos, con un bloqueo temporal que va creciendo, y 100 peticiones por minuto por token en el resto de la API. Pedir un CAPTCHA después de varios fallos. Poner un tenant_id obligatorio en todas las tablas para que la separación de clientes venga desde el modelo de datos. Hacer una sesión de modelado de amenazas con STRIDE antes de cada función nueva.	Media	2 días el rate limiting y 1 día la primera sesión de modelado

#	Vulnerabilidad	Contramedida técnica concreta	Prioridad	Esfuerzo estimado
4	Security Misconfiguration, A05:2021	Averiguar qué corre en el puerto 8080. Si es la API, dejarla escuchando solo en 127.0.0.1, poner Nginx delante en el 443 con TLS y cerrar el 8080 al público con el firewall del proveedor y con ufw. Dejar el 22 abierto solo para la IP de una VPN o de un bastión, y en sshd_config poner PasswordAuthentication no y PermitRootLogin no, entrando solo con llaves SSH. Instalar fail2ban y repetir un escaneo con nmap desde afuera para comprobar que solo responde el 443.	Alta	4 horas
5	Vulnerable and Outdated Components, A06:2021	Actualizar las 12 dependencias con CVE usando npm audit fix y a mano donde haga falta, agregar npm audit al pipeline para que falle con severidad alta o crítica, usar npm ci con package-lock.json y activar Dependabot o Renovate. Armar un servidor nuevo con Ubuntu 22.04 LTS o 24.04 LTS, con unattended-upgrades para los parches de seguridad, y pasar la aplicación. Migrar PostgreSQL 11.2 a una versión con soporte, como la 17, con pg_upgrade o con dump y restore, probado antes en un ambiente de pruebas y con respaldo. Dejar una ventana fija de parches cada mes.	Alta	3 días las dependencias y 3 semanas el servidor y la base de datos
6	Identification and Authentication Failures, A07:2021	Cambiar el secreto HS256 por uno largo y aleatorio guardado en un gestor de secretos, y rotarlo cada 90 días usando el campo kid para aceptar la llave anterior por un tiempo. Bajar el token de acceso a 15 minutos y usar un refresh token de 7 días que rote en cada uso y se pueda revocar en el servidor. Guardarlos en cookies HttpOnly, Secure y SameSite Strict y no en localStorage. Pedir MFA con TOTP o WebAuthn, empezando por los administradores, y revisar las contraseñas nuevas contra una lista de contraseñas filtradas.	Alta	1 semana
7	Security Logging and Monitoring Failures, A09:2021	Registrar en JSON con pino, desde la API y Nginx, quién hizo qué, sobre qué recurso, de qué empresa y con qué resultado. Mandar los logs a un sistema central como Loki o ELK con retención de 12 meses. Registrar logins fallidos, respuestas 403 y 404 sobre documentos y accesos por SSH. Crear alertas, por ejemplo más de 20 respuestas 403 o 404 de un mismo usuario en 5 minutos o más de 10 logins fallidos por cuenta. Definir quién recibe la alerta y un tiempo máximo de respuesta, por ejemplo 4 horas, en vez de los 6 días del incidente.	Media	2 semanas, y el primer día solo con los logs de acceso de Nginx

La prioridad sale de la matriz de riesgo. Lo que quedó en rojo es Alta y lo que quedó en naranja es Media. Los esfuerzos son estimaciones mías para uno o dos desarrolladores y habría que ajustarlos con el equipo.

El orden en que lo haría es este. El primer día cierro los puertos, roto el secreto del JWT y activo los logs de acceso, que son horas de trabajo y bajan mucho el riesgo. Después arreglo el IDOR, porque ya hubo un incidente. Sigo con las dependencias, el rate limiting y el cambio de hash. Lo que toma semanas, que es el servidor nuevo, la migración de PostgreSQL y el cifrado, va en paralelo. Mientras tanto, como no hay logs del incidente #2024-047, no se puede descartar que se hayan visto documentos de cualquier cliente, y habría que hablar con legal para ver si toca avisar.

PARTE 5

Escenario A — El administrador demasiado confiado

El CTO de VaultApp contrató a un desarrollador freelance para una tarea puntual. Para agilizar el trabajo, le compartió sus propias credenciales de acceso al sistema. El freelance terminó su tarea, pero las credenciales nunca fueron revocadas. Dos semanas después, esas credenciales aparecieron filtradas en un repositorio público de GitHub.

1. ¿Qué vulnerabilidades del OWASP Top 10 están en juego en este escenario?

   La principal es A07 Identification and Authentication Failures. La credencial del CTO se compartió con otra persona, el caso no menciona MFA que la proteja, no se revocó cuando terminó la tarea y terminó en un repositorio público. OWASP mete en esa categoría el uso de credenciales escritas en el código, con CWE-798 y CWE-259.

   También está A01 Broken Access Control, porque el freelance recibió todos los permisos del CTO en vez de unos mínimos para su tarea, y eso va contra el principio de mínimo privilegio. A09 Security Logging and Monitoring Failures también aplica, porque nadie notó que esa cuenta se usaba desde lugares raros y tampoco se podría separar lo que hizo el freelance de lo que hizo el CTO. Y A04 Insecure Design, porque no hay un proceso pensado para dar y quitar accesos a terceros.

2. ¿Qué controles técnicos y organizacionales habrían prevenido esto?

   Controles técnicos. Una cuenta propia para el freelance, con un rol que solo permita lo que necesitaba y con fecha de vencimiento, por ejemplo 7 días, en lugar de la cuenta del CTO. MFA obligatorio para todos, y en especial para administradores, de modo que una contraseña filtrada no baste. Escaneo de secretos con gitleaks antes de cada commit y secret scanning de GitHub con push protection, que bloquea el push si trae una credencial. Un gestor de secretos con credenciales temporales, para que nadie

tenga que pasar una clave por chat. Y alertas por inicios de sesión desde una IP nueva o fuera de horario en las cuentas administrativas.

Controles organizacionales. Una política que prohíba compartir credenciales. Un proceso de alta y baja de accesos con un responsable y una lista de chequeo, para que cuando termine la tarea alguien revoque todo el mismo día. Una revisión de accesos cada trimestre. Un contrato con acuerdo de confidencialidad para el freelance y capacitación para el CTO y los desarrolladores.

3. ¿Este es un control preventivo, detectivo o correctivo? ¿Por qué?

   Los controles que propuse son sobre todo preventivos, porque actúan antes de que la credencial se pueda usar mal. Las cuentas individuales con permisos mínimos, el MFA, revocar el acceso al terminar y el escaneo de secretos antes del commit evitan que el problema ocurra. Las alertas por inicios de sesión raros y el escaneo de secretos sobre los repositorios son detectivos, porque no lo evitan pero avisan rápido. Y rotar la credencial, cerrar sesiones y revisar qué se hizo después de la filtración serían correctivos.

   El control que faltó, revocar la credencial al terminar la tarea, es preventivo. Si se hace después de la filtración ya sería correctivo y llegaría tarde. En VaultApp además sería más difícil, porque los tokens duran 30 días y el secreto nunca se rota. Aunque el CTO cambiara su clave, los tokens ya emitidos seguirían sirviendo, y habría que rotar el secreto del JWT para cerrar todas las sesiones.

Escenario B — La actualización que nunca llegó

Una librería usada por VaultApp tiene un CVE crítico publicado hace 3 meses (CVSS 9.8 — ejecución remota de código sin autenticación). El equipo estaba enfocado en nuevas funcionalidades y no revisó las alertas de seguridad. Hoy, un atacante automatizado detecta la versión vulnerable y compromete el servidor.

1. ¿Qué proceso debería tener VaultApp para evitar llegar a esta situación?

   Un proceso de gestión de vulnerabilidades y parches que no dependa de que alguien se acuerde. Lo armaría en cinco partes. Primero, un inventario de dependencias, o SBOM, generado en cada build, para saber qué librerías y versiones hay. Segundo, análisis automático en el pipeline con npm audit y Dependabot o Snyk, que avisen en un canal, abran el pull request de actualización y hagan fallar el build si aparece una vulnerabilidad alta o crítica. Tercero, una persona responsable que cada semana revise las alertas y las clasifique por severidad, por exposición, por si está en el catálogo KEV de CISA y por su EPSS. Cuarto, plazos por severidad, por ejemplo 72 horas para las críticas y 24 horas si ya las están explotando, 7 días para las altas, 30 para las medias y el siguiente release para las bajas. Quinto, pruebas automáticas y un ambiente de staging, con plan de reversión, para que actualizar no dé miedo.

Mientras sale el parche se usan controles temporales, como un WAF o apagar la función afectada. Con esto, una falla crítica de hace 3 meses se habría corregido en días. Hoy VaultApp tiene 12 dependencias con CVE conocidos y nadie las revisa, así que el problema ya existe.

2. ¿Cómo priorizarían este parche frente al desarrollo de nuevas funcionalidades? ¿Qué argumentos usarían ante el CEO?

   La pondría por encima de las funcionalidades nuevas y la sacaría de la discusión normal del backlog. Una vulnerabilidad crítica de ejecución remota sin autenticación, con parche publicado hace 3 meses, no es una función contra otra, es un riesgo que el servicio ya tiene y que cualquier bot puede aprovechar. La regla sería que si está en KEV o tiene exploit público se arregla ya, y las demás por su plazo. Además, como en el escenario el atacante ya entró, hoy el parche hace parte de la respuesta al incidente, que es aislar el servidor, guardar evidencia, parchear y cambiar todos los secretos, incluido el del JWT.

   Al CEO le hablaría de negocio. Uno, lo que vendemos es seguridad, así que un servidor comprometido por una falla conocida es el peor mensaje para los 80 clientes y pone en riesgo contratos y la expansión. Dos, es barato comparado con la alternativa. Actualizar una librería suele tomar uno o dos días de un desarrollador con pruebas, y una brecha cuesta respuesta a incidente, análisis forense, avisar a clientes y a la SIC, multas de hasta 2.000 salarios mínimos mensuales y clientes que se van. Tres, no es un riesgo teórico, porque los ataques a fallas con exploit público son automáticos y pasan en horas. Cuatro, atrasar las funcionalidades unos días se puede planear y recuperar, y el daño de una brecha no. Y propondría reservar un porcentaje fijo de cada sprint, por ejemplo 20 %, para mantenimiento y seguridad, y así no negociarlo cada vez.

Fuentes consultadas

OWASP, Top 10:2021, páginas de las categorías A01, A02, A03, A04, A05, A06, A07, A09 y A10. https://top10.owasp.org/2021/. Consultado el 28 de octubre de 2026.

OWASP, Top 10:2025, solo para ver los cambios de numeración. https://top10.owasp.org/2025/. Consultado el 28 de octubre de 2026.

OWASP Cheat Sheet Series, Password Storage, HTML5 Security e Insecure Direct Object Reference Prevention. https://cheatsheetseries.owasp.org. Consultados el 28 de octubre de 2026.

endoflife.date, fechas de fin de soporte de Ubuntu 18.04 y de PostgreSQL 11. https://endoflife.date/ubuntu y https://endoflife.date/postgresql. Consultado el 28 de octubre de 2026.

PostgreSQL, política de versiones y notas de la versión 11.2. https://www.postgresql.org/support/versioning/ y https://www.postgresql.org/docs/release/11.2/. Consultados el 28 de octubre de 2026.

NVD de NIST, ficha de CVE-2019-10164. https://nvd.nist.gov/vuln/detail/CVE-2019-10164. Consultada el 28 de octubre de 2026.

Congreso de Colombia, Ley 1581 de 2012, artículos 4, 17 y 23. http://www.secretariasenado.gov.co/senado/basedoc/ley_1581_2012.html. Consultada el 28 de octubre de 2026.

CISA, catálogo de vulnerabilidades explotadas conocidas, KEV. https://www.cisa.gov/known-exploited-vulnerabilities-catalog